تولید می‌کند. مقدار HZ بستگی به این دارد که هسته چگونه پیکربندی شده، همین‌طور نوع ماشین و معماری که روی آن اجرا می‌شود. متغیر مرتبط هسته jiffies می‌باشد، معرف تعداد وقفه‌های تایمر که از زمان راه‌اندازی سیستم اتفاق داده‌اند. یک پروژه‌ی برنامه‌نویسی در فصل ۲ گنجانده شده است که زمان‌سنجی را با جزئیات بیشتر در هسته‌ی لینوکس بررسی می‌نماید.

۱-۵ مدیریت منبع

چنان‌چه دیده‌ایم، سیستم‌عامل یک **مدیر منبع** می‌باشد. CPU، فضای حافظه، فضای ذخیره‌سازی فایل، و دستگاه‌های I/O از جمله‌ی منابعی هستند که سیستم‌عامل بایستی مدیریت کند.

۱-۵-۱ مدیریت پردازه

یک برنامه هیچ کاری نمی‌تواند انجام بدهد مگر این که دستورالعمل‌های آن توسط CPU اجرا شوند. همان‌گونه که ذکر شد، یک برنامه‌ی در حال اجرا، یک پردازه است. یک برنامه، همچون کامپایلر، یک پردازه است، و یک برنامه‌ی واژه‌پرداز که توسط یک کاربر بر روی یک کامپیوتر شخصی اجرا می‌شود یک پردازه است. با تشابهی، یک app چندرسانه‌ای اجتماعی روی یک دستگاه موبایل، یک پردازه است. فعلاً می‌توانید پردازه را یک نمونه از یک برنامه‌ی در حال اجرا در نظر بگیرید، اما بعدها می‌آموزید که مفهوم آن کلی‌تر است. همان‌گونه که در فصل ۳ مطرح می‌شود، می‌توان فراخوانی‌های سیستمی فراهم کرد که اجازه می‌دهند پردازه‌ها زیرپردازه‌هایی ایجاد کنند تا به‌صورت همروند اجرا داشته باشند.

یک پردازه برای انجام وظیفه‌ی خود به منابع خاصی—اعم از زمان CPU، حافظه، فایل و دستگاه‌های I/O—نیاز دارد. این منابع نوعاً در حالی که پردازه اجرا دارد به‌آن تخصیص داده می‌شوند. علاوه بر منابع منطقی و فیزیکی مختلفی که در زمان ایجاد پردازه به آن تخصیص داده می‌شوند، امکان دارد داده‌های (ورودی) مقداردهی اولیه نیز به آن منتقل شوند. برای مثال، پردازه‌ای را در نظر بگیرید که یک مرورگر وب را اجرا می‌کند که کارش، نمایش محتویات یک صفحه‌ی وب بر روی صفحه‌نمایش است. URL، به عنوان ورودی به پردازه داده می‌شود و در ادامه، پردازه دستورات و فراخوانی‌های سیستمی مناسبی را اجرا خواهد کرد تا اطلاعات مطلوب را بدست آورده و روی صفحه نمایش دهد. زمانی که پردازه خاتمه می‌یابد، سیستم‌عامل هر نوع منبعی را که قابلیت استفاده‌ی مجدد داشته باشد از پردازه پس می‌گیرد.

تأکید می‌کنیم که یک برنامه به خودی خود یک پردازه نیست. برنامه، یک موجودیت *منفعل*[1] است، مثل محتویات یک فایل که روی دیسک ذخیره شده است، حال آن که پردازه یک موجودیت *فعال*[2] است. یک پردازه‌ی تک‌نخی، یک **شمارنده‌ی برنامه** دارد که دستورالعمل بعدی اجرایی آن را مشخص می‌کند (نخ‌ها در فصل ۴ پوشش داده می‌شوند). اجرای چنین پردازه‌ای باید ترتیبی باشد. پردازنده دستورالعمل‌های پردازه را یکی پس از دیگری تا تکمیل آن اجرا می‌کند. به علاوه، در هر

1. Passive
2. Active

لحظه حداکثر یک دستورالعمل برای پردازه اجرا می‌شود. بنابراین، هرچند ممکن است که دو پردازه به برنامه‌ای یکسان منتسب باشند؛ با این وجود، دو دنباله‌ی اجرایی مجزا محسوب می‌شوند. یک پردازه‌ی چندنخی چندین شمارنده‌ی برنامه دارد که هر یک، دستورالعمل اجرایی بعدی نخ خاصی را مشخص می‌کند.

پردازه، واحد کار یک سیستم است. یک سیستم دارای مجموعه‌ای از پردازه‌هاست که برخی از آنها پردازه‌های سیستم‌عامل (کد سیستم را اجرا می‌کنند) و بقیه، پردازه‌های کاربر (کد کاربر را اجرا می‌کنند) هستند. تمام این پردازه‌ها بالقوه می‌توانند اجرای همروند داشته باشند—برای مثال، با مالتی‌پلکس بر روی فقط یک پردازنده—یا به‌صورت موازی در میان هسته‌های متعدد CPU.

سیستم‌عامل در ارتباط با مدیریت پردازه، مسئول انجام فعالیت‌های زیر است:

- ایجاد و حذف هر دوی پردازه‌های کاربر و سیستم
- زمانبندی پردازه‌ها و نخ‌ها بر روی پردازنده‌ها
- تعلیق و از سرگیری پردازه‌ها
- ارائه‌ی راهکارهایی برای همگامی پردازه
- ارائه‌ی راهکارهایی برای ارتباط پردازه

تکنیک‌های مدیریت پردازه را در فصول ۳ تا ۷ بحث خواهیم کرد.

۱-۵-۲ مدیریت حافظه

همان‌گونه که در بخش ۱-۲-۲ بحث کردیم، حافظه‌ی اصلی دارای یک نقش محوری در عملیات سیستم کامپیوتری مدرن است. حافظه‌ی اصلی آرایه‌ی بزرگی از بایت‌هاست که اندازه‌ی آن از صدها هزار تا بیلیون‌ها بایت تغییر می‌کند. هر کلمه یا بایت، آدرس خاص خود را دارد. حافظه‌ی اصلی یک انبار دسترسی سریع داده‌هاست که میان پردازنده و دستگاه‌های I/O به طور مشترک استفاده می‌شود. CPU در طی دور واکشی دستورالعمل، دستورالعمل‌ها را از حافظه‌ی اصلی می‌خواند و در طی دور واکشی داده‌ها (با معماری von Neumann)، داده‌ها را توأمان از حافظه خوانده و می‌نویسد. چنان که پیشتر مطرح شد، حافظه‌ی اصلی، تنها دستگاه حافظه‌ی بزرگی است که پردازنده می‌تواند آدرس‌دهی کرده و مستقیماً دسترسی نماید. برای مثال، برای این که پردازنده داده‌های دیسک را پردازش کند، بایستی نخست داده‌ها توسط فراخوانی‌های I/O تولید شده توسط پردازنده به حافظه‌ی اصلی منتقل شوند. به همان شیوه، برای این که پردازنده دستورالعمل‌ها را اجرا کند بایستی در حافظه قرار بگیرند.

برنامه برای اجرا بایستی به آدرس‌های مطلقی[1] نگاشته شده و داخل حافظه بار شود. همچنان که برنامه اجرا می‌شود، دستورالعمل‌ها و داده‌های برنامه را با تولید این آدرس‌های مطلق از حافظه دستیابی می‌کند. سرانجام، برنامه خاتمه یافته، فضای حافظه‌ی آن آزاد اعلام می‌شود و برنامه‌ی بعدی می‌تواند بارگذاری و اجرا شود.

1. Absolute addresses

کامپیوترهای همه‌منظوره به منظور بهبود هر دوی بهره‌وری پردازنده و سرعت پاسخگویی به کاربران، بایستی چندین برنامه را در داخل حافظه نگه بدارند که این امر، نیاز مدیریت حافظه را مطرح می‌کند. در این راستا، طرح‌های مدیریت حافظه‌ی متفاوت زیادی استفاده می‌شوند. این طرح‌ها از رویکردهای گوناگونی استفاده می‌کنند و کارآمدی یک الگوریتم خاص به شرایط استفاده از آن بستگی دارد. در انتخاب یک طرح مدیریت حافظه برای یک سیستم خاص بایستی عوامل زیادی—به ویژه طراحی *سخت‌افزار* سیستم—را به حساب بیاوریم. هر الگوریتم، پشتیبانی سخت‌افزاری خاص خود را نیاز دارد.

سیستم‌عامل در ارتباط با مدیریت حافظه، مسئول انجام فعالیت‌های زیر است:

- حفظ رد بخش‌هایی از حافظه که در حال حاضر استفاده می‌شوند و کدام پردازه از آنها استفاده می‌کند
- تخصیص و باز پس‌گیری فضای حافظه در مواقع لازم
- تصمیم‌گیری درباره‌ی این که کدام پردازه‌ها (یا بخش‌هایی از پردازه‌ها) و داده‌ها قرار است به داخل و خارج حافظه جابه‌جا شوند

تکنیک‌های مدیریت حافظه در فصل ۹ و فصل ۱۰ بحث می‌شوند.

۱-۵-۳ مدیریت سیستم‌فایل

سیستم‌عامل برای راحتی کاربران سیستم کامپیوتر، دید منطقی یکشکلی از ذخیره‌سازی اطلاعات فراهم می‌کند. سیستم‌عامل فارغ از ویژگی‌های فیزیکی دستگاه‌های ذخیره‌سازی خود، واحد منطقی ذخیره‌سازی، یعنی **فایل** را تعریف می‌کند. سیستم‌عامل، فایل‌ها را روی رسانه‌ی فیزیکی می‌نگارد و این فایل‌ها را از طریق دستگاه‌های ذخیره‌سازی دستیابی می‌کند.

یکی از مؤلفه‌های سیستم‌عامل که بیشتر در معرض دید قرار دارد، مدیریت فایل است. کامپیوترها می‌توانند اطلاعات را روی انواع متعدد متفاوتی از رسانه‌های فیزیکی ذخیره کنند. حافظه‌ی ثانویه، رایج‌ترین رسانه‌ها هستند، اما حافظه‌ی ثالث نیز امکان‌پذیر است. هر یک از این رسانه‌ها مشخصات و سازماندهی فیزیکی خاص خود را دارند. اکثراً توسط یک دستگاه کنترل می‌شوند، همچون دیسک‌گردان که آن هم مشخصه‌های یکتای خاص خود را دارد. این ویژگی‌ها شامل سرعت دسترسی، ظرفیت، نرخ انتقال داده‌ها و روش دسترسی (ترتیبی یا تصادفی) هستند.

فایل، مجموعه‌ای از اطلاعات مرتبط است که توسط ایجادکننده‌اش تعریف می‌شود. فایل‌ها عمدتاً برنامه‌ها (هر دو شکل اصلی[1] و مقصد[2]) و داده‌ها را نمایش می‌دهند. فایل‌های داده‌ها می‌توانند عددی، الفبایی، الفباعددی یا دودویی باشند. فایل‌ها می‌توانند قالب خاصی نداشته (برای مثال، فایل‌های متنی) و یا ممکن است قالب مشخصی (برای مثال، فیلدهای ثابت همچون فایل صوتی mp3) داشته باشند. روشن است که مفهوم یک فایل، بی‌نهایت کلی است.

1. Source
2. Object

سیستم‌عامل، مفهوم انتزاعی فایل را با مدیریت رسانه‌ی ذخیره‌سازی انبوه و دستگاه‌هایی که آن‌ها را کنترل می‌کنند، پیاده‌سازی می‌کند. به علاوه، فایل‌ها برای سادگی استفاده، طبق معمول به صورت دایرکتوری‌هایی سازماندهی می‌شوند. بالاخره، زمانی که چندین کاربر به فایل‌ها دسترسی دارند، شاید این کنترل مطلوب باشد که، کدام کاربر می‌تواند به یک فایل و به چه نحوی دسترسی داشته باشد (برای مثال، دسترسی read، write و append).

سیستم‌عامل در ارتباط با مدیریت فایل، مسئول انجام فعالیت‌های زیر است:

- ایجاد و حذف فایل‌ها
- ایجاد و حذف دایرکتوری‌ها جهت سازماندهی فایل‌ها
- پشتیبانی دستورالعمل‌های ابتدایی دستکاری فایل‌ها و دایرکتوری‌ها
- نگاشت فایل‌ها به روی ذخیره‌سازی انبوه
- پشتیبان‌گیری فایل‌ها روی رسانه‌ی ذخیره‌سازی پایدار (نامیرا)

تکنیک‌های مدیریت فایل در فصل ۱۳، فصل ۱۴ و فصل ۱۵ بحث می‌شوند.

۱-۵-۴ مدیریت ذخیره‌سازی انبوه

همان‌گونه که قبلاً دیده‌ایم، سیستم کامپیوتری بایستی به منظور پشتیبانی حافظه‌ی اصلی، حافظه‌ی ثانویه را فراهم کند. اکثر سیستم‌های کامپیوتری مدرن از دستگاه‌های HDD و NVM به عنوان رسانه‌ی ذخیره‌سازی اصلی برخط، برای هر دوی برنامه‌ها و داده‌ها استفاده می‌کنند. اکثر برنامه‌ها – اعم از کامپایلرها، مرورگرهای وب، واژه‌پردازها، و بازی‌ها[1]—روی این دستگاه‌ها ذخیره می‌شوند تا این که داخل حافظه بار شوند. سپس، برنامه‌ها از این دستگاه‌ها توأمان به عنوان مبدأ و مقصد پردازش خود استفاده می‌کنند. از این رو، مدیریت درست حافظه‌ی ثانویه در سیستم کامپیوتری از اهمیت محوری برخوردار است. سیستم‌عامل در ارتباط با مدیریت حافظه‌ی ثانویه، مسئول انجام فعالیت‌های زیر است:

- سوارکردن[2] و برداشتن[3]
- مدیریت فضای آزاد
- تخصیص حافظه
- زمانبندی دیسک
- پارتیشن‌بندی
- حفاظت

چون حافظه‌ی ثانویه با دفعات بیشتر و زیاد استفاده می‌شود، بایستی به صورت کارآمد استفاده شود. سرعت کل عملیات یک کامپیوتر می‌تواند تحت‌تأثیر سرعت زیرسیستم حافظه‌ی ثانویه و الگوریتم‌های مرتبط با این زیرسیستم قرار بگیرد.

1. Formatters
2. Mounting
3. Unmounting

در عین حال، استفاده‌های فراوانی از ذخیره‌سازی وجود دارند که کندتر و ارزانتر (و گاهاً دارای ظرفیت بیشتر) از حافظه‌ی ثانویه هستند. پشتیبان‌های داده‌های دیسک، ذخیره‌سازی داده‌هایی که به ندرت استفاده می‌شوند و ذخیره‌سازی آرشیو بلندمدت، مثال‌هایی از این دست هستند. نوارگردان‌های مغناطیسی و نوارهای آنها و دیسک‌گردان‌های CD، DVD و Blu-ray و صفحات گرامافون، دستگاه‌های **حافظه‌ی ثالث**[1] معمول هستند.

حافظه‌ی ثالث در کارایی سیستم نقش حیاتی ندارد، اما باز هم بایستی مدیریت شود. بعضی سیستم‌عامل‌ها این کار را عهده‌دار هستند، در حالی که سیستم‌های دیگری هم هستند که مدیریت حافظه‌ی ثانویه را بر عهده‌ی برنامه‌های کاربردی رها می‌کنند. برخی از کارکردهایی که سیستم‌عامل‌ها می‌توانند در این راستا فراهم کنند عبارتند از: سوارکردن و برداشتن رسانه از دستگاه‌ها، تخصیص و رهاسازی دستگاه‌ها برای استفاده‌ی انحصاری پردازه‌ها و امکان نقل مکان داده‌ها از حافظه‌ی ثانویه به حافظه‌ی ثالث.

تکنیک‌های مدیریت حافظه‌ی ثانویه و حافظه‌ی ثالث در فصل ۱۱ بحث می‌شوند.

۱-۵-۵ مدیریت کَش

نهان‌سازی[2] یک اصل مهم سیستم‌های کامپیوتری است. نحوه‌ی کار آن به این صورت است. اطلاعات قاعدتاً در برخی سیستم‌های ذخیره‌سازی (نظیر حافظه‌ی اصلی) نگه‌داری می‌شوند. در زمانی که استفاده دارند، موقتاً به یک سیستم حافظه‌ی پرسرعت – نهانی—کپی می‌شوند. زمانی که به بخش خاصی از اطلاعات نیاز داریم، ابتدا بررسی می‌کنیم که آیا در کَش هست یا خیر. اگر آنجا بود، مستقیماً از اطلاعات داخل کَش استفاده می‌کنیم، اگر نباشد، از اطلاعات اصلی استفاده می‌کنیم و یک کپی از آنها را با این فرض که دوباره به زودی از آنها استفاده خواهیم کرد در کَش قرار می‌دهیم.

به علاوه، ثبات‌های برنامه‌پذیر داخلی، نظیر ثبات‌های ایندکس، کَش پرسرعتی برای حافظه‌ی اصلی فراهم می‌کنند. برنامه‌نویس (یا کامپایلر) با پیاده‌سازی الگوریتم‌های تخصیص ثبات و جایگزینی ثبات در رابطه با نگهداری اطلاعات در ثبات‌ها و یا در حافظه‌ی اصلی تصمیم‌گیری می‌کند.

کَش‌های دیگری هستند که کاملاً در سخت‌افزار پیاده‌سازی می‌شوند. برای نمونه، اکثر سیستم‌ها یک کش دستورالعمل دارند که دستورالعمل‌های اجرایی مورد انتظار بعدی را نگه می‌دارد. پردازنده با نداشتن این حافظه نهان مجبور خواهد شد که چندین دور انتظار بکشد تا دستورالعمل از حافظه‌ی اصلی واکشی شود. به دلایل مشابهی، اکثر سیستم‌ها در سلسله‌مراتب ذخیره‌سازی داده‌های خود، یک یا چند کش پرسرعت دارند. از آن جایی که کَش‌های کاملاً سخت‌افزاری، خارج از کنترل سیستم‌عامل هستند، در این کتاب به کش‌هایی که کاملاً سخت‌افزاری هستند، نمی‌پردازیم.

1. Tertiary storage
2. Caching

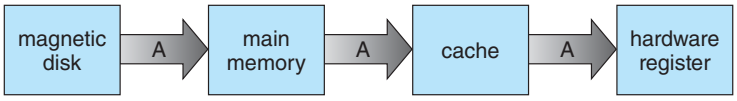
چون کَش‌ها، اندازه‌ی محدودی دارند، **مدیریت کَش** یک مسأله‌ی مهم طراحی است. دقت در انتخاب اندازه‌ی کَش و سیاست جایگزینی مناسب، موجب افزایش قابل توجهی در کارایی می‌شود، چنان‌چه با بررسی شکل ۱-۱۴ می‌بینید. الگوریتم‌های جایگزینی مختلف کَش‌های با کنترل نرم‌افزاری در فصل ۱۰ بحث می‌شوند.

جابه‌جایی اطلاعات میان سطوح سلسله‌مراتب حافظه، بسته به طراحی سخت‌افزار و کنترل نرم‌افزار سیستم‌عامل می‌تواند صریح و یا ضمنی باشد. برای نمونه، انتقال داده‌ها از کش به پردازنده و ثبات‌ها معمولاً یک کار سخت‌افزاری است و سیستم‌عامل دخالتی در آن ندارد. در مقایسه، انتقال داده‌ها از دیسک به حافظه معمولاً توسط سیستم‌عامل کنترل می‌شود.

Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

شکل ۱-۱۴ مشخصه‌های انواع مختلف حافظه.

در ساختار ذخیره‌سازی سلسله‌مراتبی امکان دارد که داده‌های یکسانی در سطوح متفاوتی از سیستم ذخیره‌سازی ظاهر شوند. برای مثال، فرض کنید متغیر صحیح A که قرار است ۱ واحد افزایش یابد در فایل B قرار دارد و فایل B روی دیسک سخت مقیم است. عملیات افزایش، نخست با صدور یک عملیات I/O می‌خواهد تا بلاک حاوی متغیر A به حافظه‌ی اصلی آورده شود. این عملیات با کپی A به کش و یک ثبات داخلی دنبال می‌شود. بنابراین، کپی A در چندین مکان وجود دارد: دیسک سخت، حافظه‌ی اصلی، کش و یک ثبات داخلی (شکل ۱-۱۵ را ببینید). همین که افزایش در ثبات داخلی صورت بگیرد، مقدار A در سیستم‌های مختلف ذخیره‌سازی متفاوت خواهد شد. تنها بعد از این که مقدار جدید A دوباره از ثبات داخلی در دیسک سخت نوشته می‌شود، مقدار A یکسان می‌گردد.



شکل ۱-۱۵ نقل مکان مقدار متغیر صحیح A از دیسک به ثبات.

این مسأله در یک محیط محاسباتی که تنها یک پردازه در هر زمان اجرا می‌شود، هیچ مشکلی ایجاد نمی‌کند، چون هرگونه دسترسی به متغیر صحیح A، همواره به کپی بالاترین سطح

سلسله‌مراتب خواهد بود. با این همه، در یک محیط چندوظیفگی که پردازنده میان پردازه‌های متعدد دست به دست می‌چرخد، بایستی دقت زیادی مبذول شود تا تضمین گردد، اگر چندین پردازه بخواهند A را دستیابی کنند، در این صورت هر یک از پردازه‌ها، اخیرترین مقدار به روز شده‌ی A را به‌دست می‌آورند.

این وضعیت در یک محیط چندپردازنده که هر کدام از پردازنده‌ها علاوه بر داشتن ثبات‌های داخلی، یک کَش محلی هم دارند (به شکل ۱-۸ مراجعه کنید) به مراتب پیچیده‌تر می‌شود. در چنین محیطی، کپی A ممکن است همزمان در کش‌های متعددی وجود داشته باشد. از آن جایی که CPUهای مختلف می‌توانند همگی اجرای موازی داشته باشند، بایستی اطمینان بدهیم که به روز شدن مقدار A در یکی از کَش‌ها، بلافاصله به تمامی کَش‌هایی که مقدار A در آنها قرار دارد منعکس می‌شود. این وضعیت، **شفافیت کش**[1] نامیده می‌شود و معمولاً یک مسأله‌ی سخت‌افزاری است (پایین‌تر از سطح سیستم‌عامل رسیدگی می‌شود).

در محیط توزیع‌شده، حتی وضعیت از این هم پیچیده‌تر می‌شود. در چنین محیطی، کپی‌های (یا نسخه‌های[2]) متعددی از فایل یکسانی می‌تواند در کامپیوترهای متفاوتی که از نظر مکانی توزیع‌شده هستند، وجود داشته باشد. از آن جایی که نسخه‌های مختلف می‌توانند به طور همروند، دسترسی و به روز شوند، بایستی اطمینان بدهیم زمانی که نسخه‌ای در یک جا به روز می‌شود، سایر نسخه‌ها نیز در اسرع وقت به روز می‌گردند. شیوه‌های مختلفی برای رسیدن به این اطمینان وجود دارند که در فصل ۱۹ بحث می‌کنیم.

۱-۵-۶ مدیریت سیستم I/O

یکی از اهداف سیستم‌عامل این است که ویژگی‌های خاص دستگاه‌های سخت‌افزاری به‌خصوصی را از کاربر مخفی نگه بدارد. برای مثال، در یونیکس، جزئیات و خصوصیات دستگاه‌های I/O از پیچیدگی و درون خود سیستم‌عامل، توسط **زیرسیستم I/O** مخفی می‌شوند. زیرسیستم I/O شامل چندین مؤلفه است که عبارتند از:

- مؤلفه‌ی مدیریت حافظه که شامل بافرسازی، کش‌سازی و اسپولینگ می‌باشد
- رابط عمومی درایور دستگاه
- درایورهایی برای دستگاه‌های سخت‌افزاری خاص

تنها درایور دستگاه است که به خصوصیات دستگاه خاصی که به آن گمارده شده است آگاه است.

پیشتر در این فصل، درباره‌ی چگونگی استفاده از راه‌اندازهای وقفه و درایورهای دستگاه برای برپایی مؤثر زیرسیستم‌های I/O بحث کردیم. در فصل ۱۲، درباره‌ی چگونگی برقراری ارتباط زیرسیستم I/O با سایر مؤلفه‌های سیستم، مدیریت دستگاه‌ها، نقل و انتقال داده‌ها و تشخیص تکمیل I/O بحث می‌کنیم.

---

1. Cache coherency
2. Replica